

Projet Binairo - documentation

Auteurs : Louis Pedelaborde, Romeo Mugnier de Almeida, Raphael Pical. **Etat : 1 octobre 2026.**
Référence technique : donnée Binairo 2026.2 du 7 septembre 2026. Cette version décrit les sources actuelles ; la partie LabVIEW reste à construire et valider.

Environnement et vérifications

Elément	Environnement effectivement utilisé
Machine du test C	Machine locale MacBookAir10,1, processeur Apple M1, architecture arm64
Système	macOS 27.0.1, build 26A434
Compilateur C	Apple Clang 21.0.0, clang-2100.3.34.2 ; outils Command Line Tools
Validation du 1 octobre	46 essais C ciblés : dimensions, pixels, seuils, fichiers et échecs d'allocation ; tous réussis
MATLAB	Non exécuté dans cet environnement ; version de l'installation cible à relever lors du test
LabVIEW	Version annoncée par le groupe : 2025 Q3, menus anglais ; VIs non exécutés ici
VM Windows et compilateur	Versions et configuration à relever sur la VM lors de la compilation et des essais

La compilation utilise C11 et les avertissements :

```
cc -std=c11 -Wall -Wextra -Wpedantic -Werror -Wvla main.c reconnaissance.c -o OCR
```

Le programme C a aussi été vérifié avec contrôle des accès mémoire et des comportements indéfinis. Une surveillance des allocations confirme leur libération dans les 46 essais, y compris lorsque chacune des trois allocations échoue. Les tests, leurs exécutables et leurs données ne font pas partie des sources à remettre.

Fichiers C présents dans GitHub

Fichier dans c/	Rôle
main.c	Arguments, chemin du résultat, reconnaissance puis écriture ; code de retour et message d'erreur
reconnaissance.c	Lecture binaire, validation, allocation des pixels, comparaison aux modèles0/1
reconnaissance.h	Types, prototypes publics et codes de retour
parametres_ocr.h	Dimensions10..100 ; limite du chemin1023 caractères
FontRasterized_0_1.h	Police32×32 fournie par le professeur, conservée sans modification avec ses crédits

Les sources du groupe portent les noms des trois auteurs. Le fichier de police conserve l'en-tête de son auteur d'origine. Pour les VIs à créer : renseigner les auteurs, le but, les entrées/sorties et les erreurs dans VI Properties > Documentation. Le script solve.m généré possède aussi un en-tête d'auteurs.

Fonctionnement et limites

Le C reçoit le chemin de Cell.bin et les trois seuils vide/0/1 dans 0..100. Une paire de guillemets encore présente autour du chemin est retirée. Sous Windows, les chemins avec espaces doivent être groupés par des guillemets doubles ; des apostrophes seules ne le permettent pas dans cmd.

Cell.bin contient largeur puis hauteur, deux entiers non signés 32 bits petit-boutistes, suivis de largeur×hauteur octets : 0=blanc, 1=noir, ligne après ligne. Les deux dimensions doivent être dans 10..100. La cellule peut être rectangulaire ; une grille Binaire doit, elle, être carrée et de taille paire.

Les dimensions sont contrôlées avant leur produit et avant malloc. Une valeur négative encodée en U32, comme 4294967196 pour -100, ou une taille 10000 est rejetée. Le tableau de pixels utilise au plus 10000 octets. Les chemins sont alloués à leur taille exacte, au plus 1024 octets avec le caractère nul. Les trois blocs alloués directement par le programme totalisent au plus 12048 octets ; les tampons internes de la bibliothèque C ne sont pas inclus. Il n'y a ni VLA ni tableau fixe du programme dépassant 100 éléments.

Le test du vide précède la reconnaissance. Pour chaque chiffre, le modèle 32×32 est déplacé sur la cellule ; les pixels blancs et noirs identiques sont comptés. Un chiffre est admissible si son score atteint son seuil. La plus grande marge score-seuil est retenue ; 0 est conservé à égalité. Cette règle suit la procédure détaillée p18 de la donnée ; sa contradiction avec le résumé p10 reste à clarifier auprès du professeur.

CellValue.txt est écrit à côté de l'entrée, en mode texte, seulement après une reconnaissance réussie. Exemple : d: '0' , 97.167969% suivi d'une fin de ligne. Une case vide produit -2 et 0%. Cette valeur -2 est un symbole OCR, pas un code de retour. Un échec avant ouverture conserve l'ancien résultat ; un échec pendant l'écriture peut laisser un fichier incomplet. Le consommateur doit toujours vérifier le code de retour.

Codes de retour OCR et messages

Code	Signification et cas couverts
0	Succès ; le résultat courant peut être lu
2	Mauvais nombre d'arguments, chemin vide/trop long/réservé, seuil invalide
3	En-tête incomplet ; dimension hors 10..100 ; pixel manquant/en trop ; octet autre que 0/1
4	Fichier d'entrée absent/inaccessible, erreur de lecture/fermeture, allocation impossible
5	Case non vide trop petite pour les modèles ou aucun chiffre atteignant son seuil
6	Impossible d'ouvrir, écrire, vider le tampon ou fermer le résultat

Le message sur stderr commence par OCR E<code>: et précise la cause. Ainsi E3 distingue par son texte « dimensions ... hors des bornes », « nombre de pixels inférieur ... » et « nombre de pixels supérieur ... ». Les numéros -2/-3 rapportés oralement ne sont pas encore confirmés comme obligatoires : cette table décrit les codes effectivement implémentés. Si une liste officielle impose d'autres valeurs, cette correspondance et les tests devront être adaptés.

MATLAB et fichiers d'échange

Fichier dans matlab/	Rôle
SolveBinaire.m	Déductions puis essais récursifs 0/1 ; contient DirectValues, ParcourirCases, CheckValidMove, CheckVectorOk, CheckVectorUniqueOk et printGrid
FormatGrilleValide.m	Type, dimensions carrées/paires et valeurs 0/1/NaN
GrilleBinaireValide.m	Comptes, suites interdites et unicité des lignes/colonnes complètes
CreerFigureBinaire.m	Grille, chiffres, nom de fichier, date et message d'erreur
CheminPdfBinaire.m	Nom du PDF à partir du PNG
DisplayBinaire.m	Export en PDF et fermeture de la figure
DispBinaire.m	Nom compatible avec l'appel présent dans la donnée ; appelle DisplayBinaire

Le solveur répète les déductions jusqu'à stabilisation puis essaie 0 et 1 dans une case vide. Les échecs déclenchent le retour à la branche précédente. Le parcours des cases est divisé en deux pour limiter sa profondeur sans utiliser de boucle. Les fonctions communes à l'affichage restent séparées. Aucun résultat d'exécution MATLAB n'est revendiqué dans cette version.

La recette prévoit BinaireSolver.vi comme VI principal. Il lit le PNG, détecte les lignes/colonnes, extrait chaque cellule, écrit Cell.bin, lance OCR et construit la matrice. Il génère solve.m, qui appelle directement SolveBinaire et DisplayBinaire, puis vérifie le PDF. Les VIs sont à construire : leur présence dans la recette ne signifie pas qu'ils existent déjà dans le dépôt.

Fichier produit à l'exécution	Utilisation
Cell.bin	Cellule courante transmise de LabVIEW au C
CellValue.txt	Symbole et score transmis du C à LabVIEW
solve.m	Matrice B et appels MATLAB générés par LabVIEW selon les options
NomDuPNG.pdf	Résultat graphique, indices noirs gras et valeurs ajoutées bleues grasses

Un seul traitement doit utiliser le dossier d'exécution à la fois. Après une erreur C ou MATLAB, LabVIEW doit propager l'erreur et ne pas réutiliser un ancien résultat. Le script et les résultats sont créés dans le dossier d'exécution inscriptible ; aucun chemin personnel absolu n'est intégré aux sources.

A terminer avant la remise

Compiler et tester sur la VM ; relever les versions réelles de Windows, du compilateur, de MATLAB et de LabVIEW. Construire les VIs et renseigner leurs auteurs. Vérifier les quatre combinaisons Résoudre/Ouvrir PDF avec le lancement réel, ainsi que le rendu PDF et les erreurs. Ajouter au rendu les fichiers projet, l'exécutable de la plateforme cible, les VIs et les résultats demandés par la donnée. Le dépôt de sources actuel ne constitue pas encore une remise complète de la partie LabVIEW.